Experiment - 3

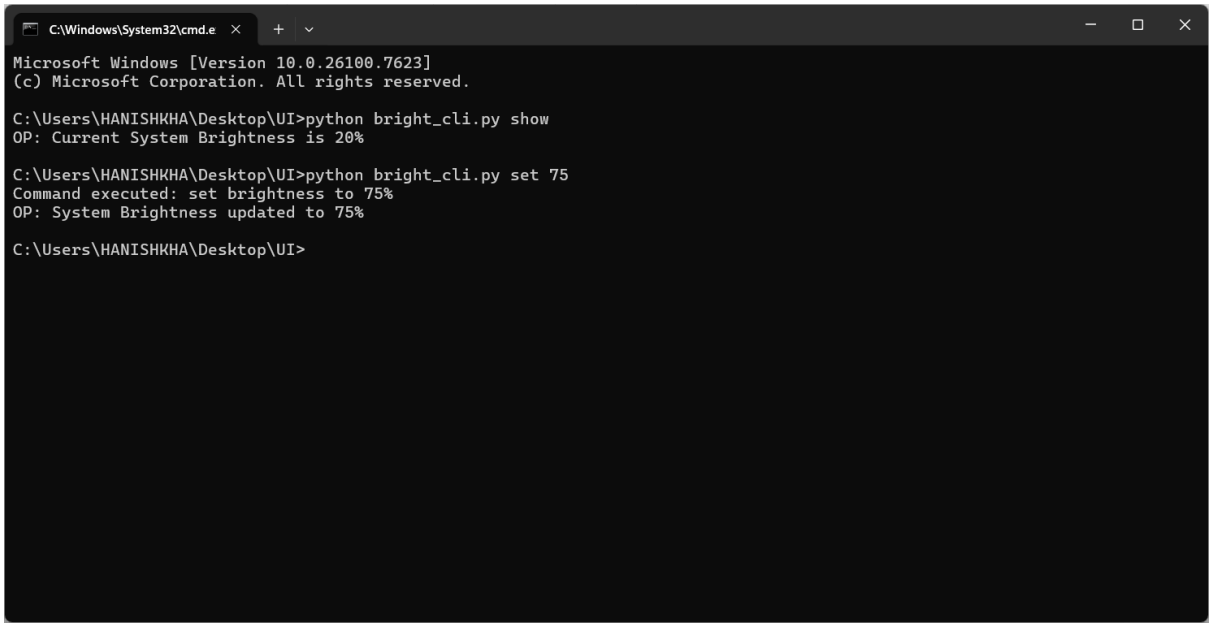
Name: N Hanishkha Roll No: 240701161

1) Commmand Line Interface (CLI)

PYTHON CODE:

```
import sys
import screen_brightness_control as sbc
def manage_brightness_cli():
    if len(sys.argv) < 2:
        print("Usage: python brightness_cli.py <command> [value]")
        print("Commands: set, show")
        return
    command = sys.argv[1].lower()
    if command == "set" and len(sys.argv) > 2:
        level = int(sys.argv[2])
        sbc.set_brightness(level) # Real system change
        print(f"Command executed: set brightness to {level}%")
        print(f"OP: System Brightness updated to {level}%")
    elif command == "show":
        current = sbc.get_brightness()[0] # Real system check
        print(f"OP: Current System Brightness is {current}%")
    else:
        print("Invalid command. Use 'set <value>' or 'show'.")
if __name__ == "__main__":
    manage_brightness_cli()
```

OUTPUT:



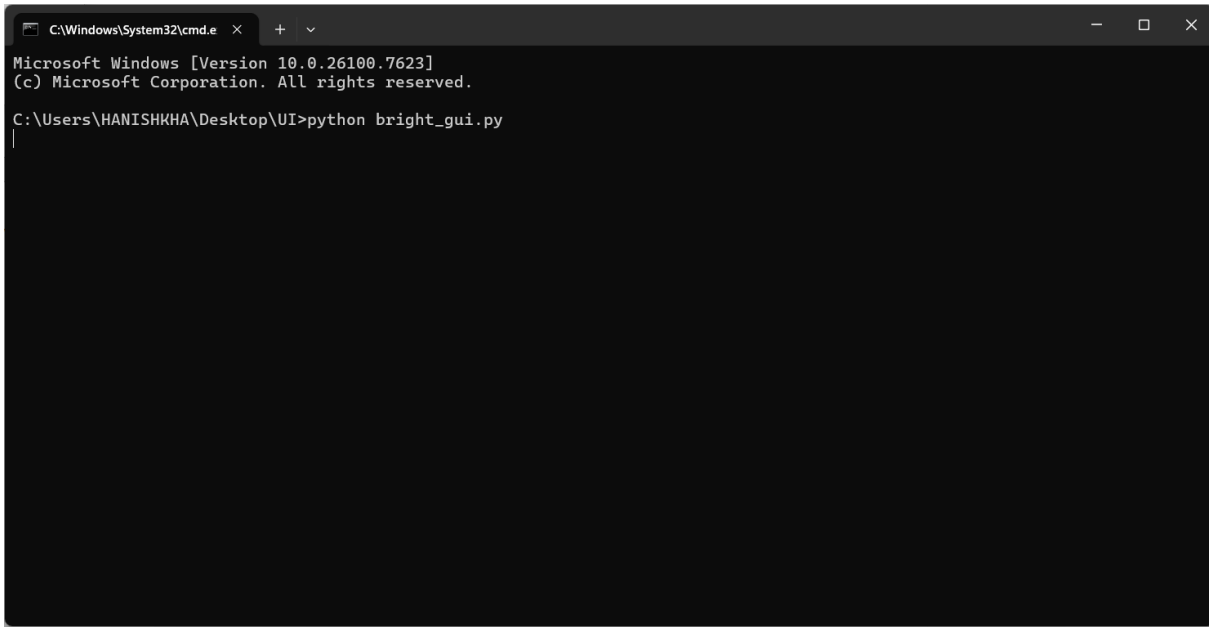
2)Graphical User Interface (GUI)

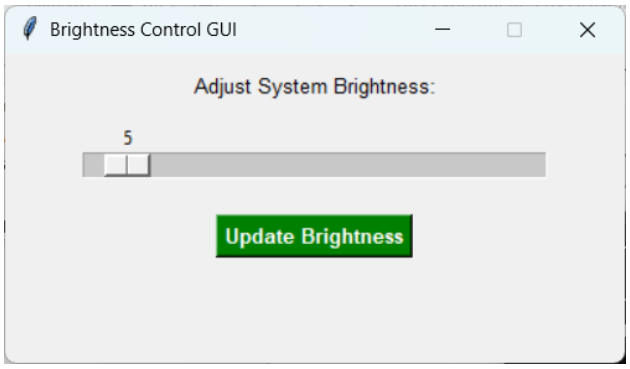
PYTHON CODE:

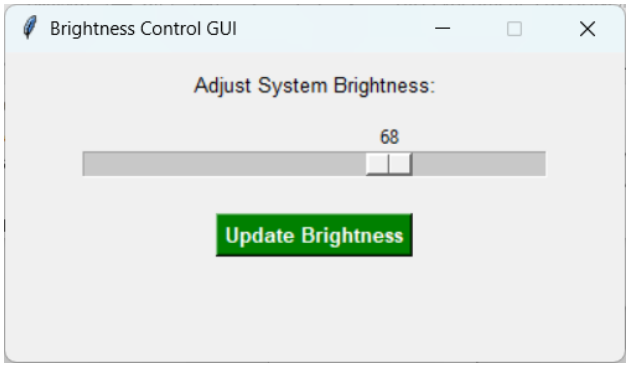
```
import tkinter as tk
from tkinter import messagebox
import screen_brightness_control as sbc
def apply_brightness_gui():
    level = slider.get()
    try:
        sbc.set_brightness(level)
        messagebox.showinfo("Output", f"OP: System Brightness adjusted to {level}%")
    except Exception as e:
        messagebox.showerror("Error", f"Failed to set brightness: {e}")
root = tk.Tk()
root.title("Brightness Control GUI")
root.geometry("400x200")
root.resizable(False, False)
# Fetch current brightness to sync the slider position
try:
    current_val = sbc.get_brightness()[0]
```

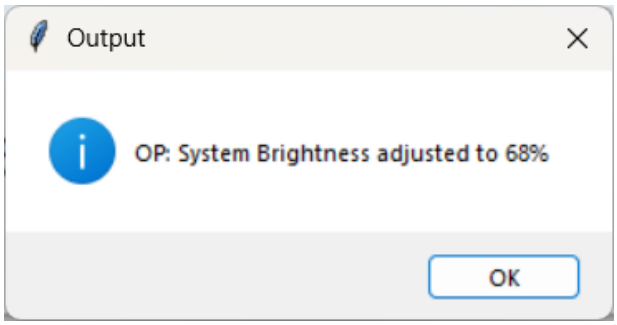
```
except:
    current_val = 50
tk.Label(root, text="Adjust System Brightness:", font=("Arial", 10)).pack(pady=10)
slider = tk.Scale(root, from_=0, to=100, orient="horizontal", length=300)
slider.set(current_val)
slider.pack()
# Styled Button (Green)
tk.Button(root, text="Update Brightness", bg="green", fg="white",
          font=("Arial", 10, "bold"), command=apply_brightness_gui).pack(pady=20)
root.mainloop()
```

OUTPUT:









3) Voice User Interface (VUI)

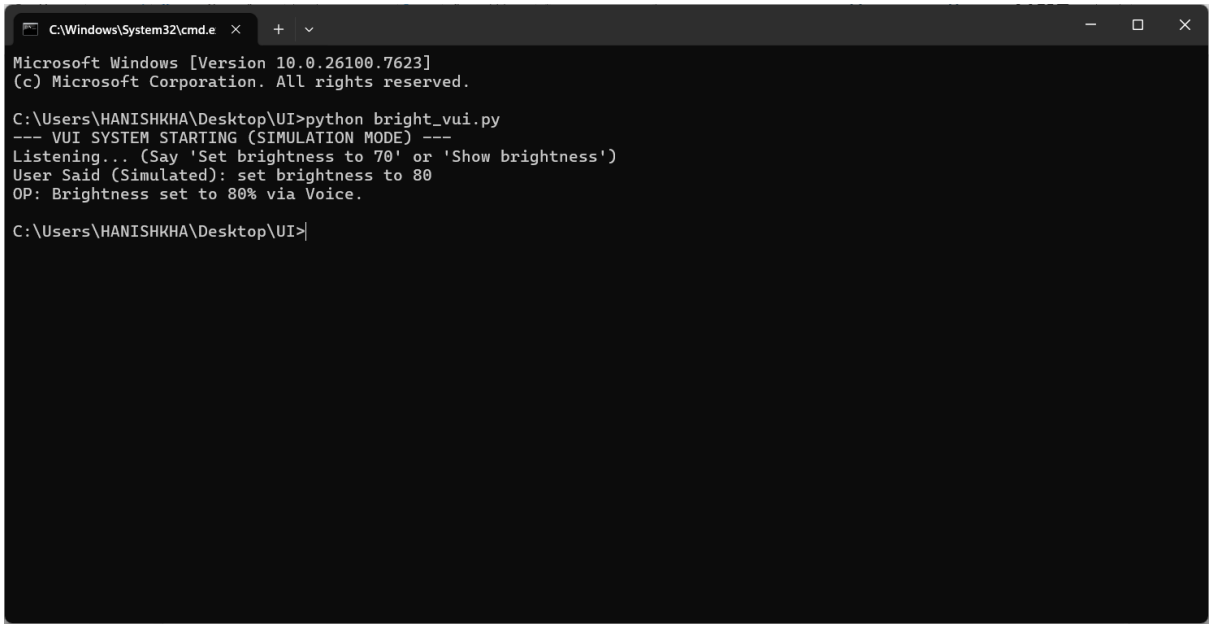
PYTHON CODE:

```
import speech_recognition as sr
import screen_brightness_control as sbc
def listen_brightness_vui():
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening... (Say 'Set brightness to 70' or 'Show brightness')")
        r.adjust_for_ambient_noise(source)
        audio = r.listen(source)

    try:
        command = r.recognize_google(audio).lower()
        print(f"User Said: {command}")
            if "set" in command:
          # Finding the number in your speech
          level = [int(s) for s in command.split() if s.isdigit()][0]
          sbc.set_brightness(level)
```

```
            print(f"OP: Brightness set to {level}% via Voice.")
        elif "show" in command or "current" in command:
            current = sbc.get_brightness()[0]
            print(f"OP: Current Brightness is {current}%.")
    except Exception as e:
        print("OP: Error - Could not process voice command.")
if __name__ == "__main__":
    listen_brightness_vui() now add the simulation code and give me
```

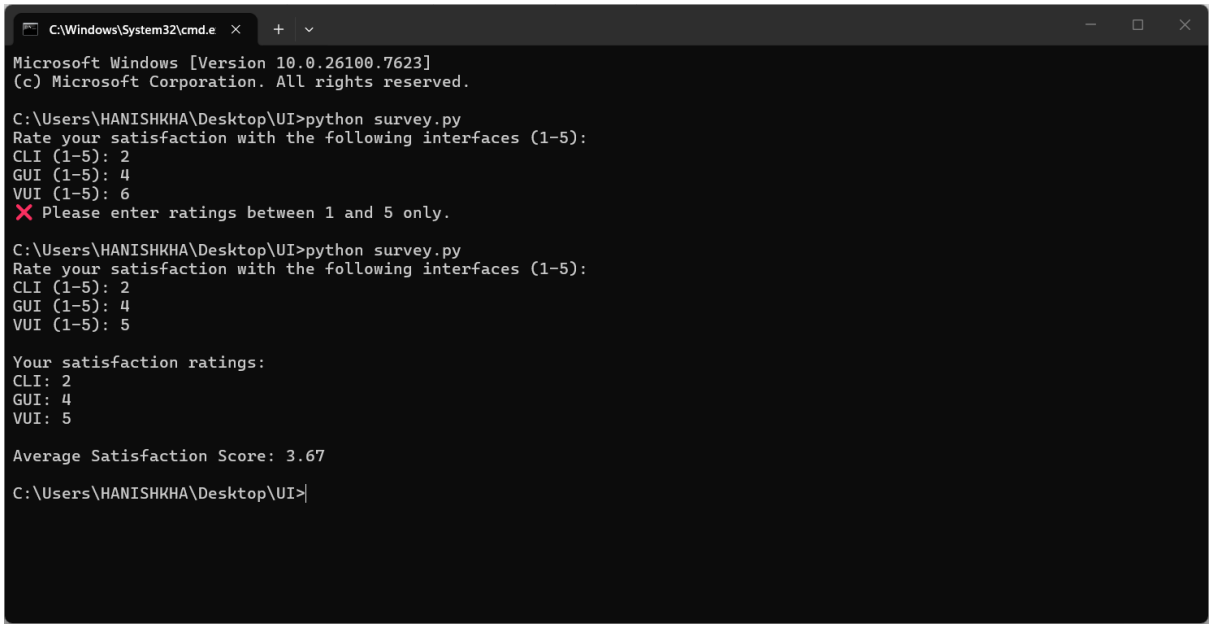
OUTPUT:



4) USER SATISFACTION COMPARISON

PYTHON CODE:

```
def survey():
    print("Rate your satisfaction with the following interfaces (1-5):")
    # Get user input for each interface
    try:
        cli_satisfaction = int(input("CLI (1-5): "))
        gui_satisfaction = int(input("GUI (1-5): "))
        vui_satisfaction = int(input("VUI (1-5): "))
        # Ensure valid ratings
```

```
        if not (1 <= cli_satisfaction <= 5 and 1 <= gui_satisfaction <= 5 and 1 <= vui_satisfaction <= 5):
            print("❌ Please enter ratings between 1 and 5 only.")
            return
        # Display the ratings
        print("\nYour satisfaction ratings:")
        print(f"CLI: {cli_satisfaction}")
        print(f"GUI: {gui_satisfaction}")
        print(f"VUI: {vui_satisfaction}")
        # Calculate the average satisfaction
        avg_satisfaction = (cli_satisfaction + gui_satisfaction + vui_satisfaction) / 3
        print(f"\nAverage Satisfaction Score: {avg_satisfaction:.2f}")
    except ValueError:
        print("⚠️ Invalid input! Please enter numbers between 1 and 5.")
# Run the survey function
if __name__ == "__main__":
    survey()
```

OUTPUT: